```
bo = Proc.new { |param| puts "You called me with #{param}" }
bo.call 99
bo.call "cat"
```

*produces:*

```
You called me with 99
You called me with cat
```

There are very slight differences between the behavior of the resulting object based on the lambda calls versus the proc calls. Specifically, the lambda values return an error if called with the wrong number of arguments, while proc will allow the call, and either truncate extra arguments or assign nil to unspecified arguments. Also, using return inside a proc will also return from the method the proc is inside, whereas using return inside a lambda will not.

## Blocks Are Closures

We said earlier that a block can use local variables from the surrounding scope. Let's look at a slightly different example of a block doing just that:

```
def n_times(thing)
  ->(n) { thing * n }
end

p1 = n_times(23)
p1.call(3) # => 69
p1.call(4) # => 92
p2 = n_times("Hello ")
p2.call(3) # => "Hello Hello Hello "
```

The method n_times uses stabby lambda syntax to return a Proc object that references the method's parameter, thing. Even though that parameter is out of scope by the time the block is called outside the method, the parameter remains accessible to the block. This is called a *closure*—variables in the surrounding scope that are referenced in a block remain accessible for the life of that block and the life of any Proc object created from that block.

Here's another example—a method that returns a Proc object that returns successive powers of 2 when called:

```
def power_proc_generator
  value = 1
  -> { value += value }
end

power_proc = power_proc_generator

puts power_proc.call
puts power_proc.call
puts power_proc.call
```

*produces:*

```
2
4
8
```

## Stabby Lambdas

Let's look at that lambda syntax a little more. You can write the following: